

CS4135 - Software Architectures Project
Assignment 3 - Part B
Event Storming & Domain Storytelling

E-Library System | University of Limerick

Team Members

Caylum Hurley - 22356363

Vivian Jently - 22079017

Oscar Zhou - 22338705

Joe Considine - 22344977

Darragh Quinn - 22359621

Daragh Downes - 22351159

Table of Contents

B1

Contents

Table of Contents	2
B1. Event Storming Outcomes	3
B1.1 Notation Key Used on the Board	3
B1.2 Big-Picture Event Flow (Session 1 - Full Domain Timeline)	3
B1.3 Hotspots & Conflicts Register	5
B1.4 Policies Discovered	7
B2. Translation of Insights	9
B2.1 Model Changes Introduced by Event Storming	9
B2.2 Backlog Items Generated	11
B3. Domain Storytelling	12
B3.1 DS-01: Student Borrows a Book	12
B3.2 DS-02: Admin Manages Book Inventory	14
B3.3 DS-03: Overdue Book - Reminder and Return Flow	15
B3.4 DS-04: Book Club - Create, Join, and Discuss	17
B4. Facilitation & Evidence	19
B4.1 Session Summary	19
B4.2 Participant Roles in Sessions	20
B4.3 Miro Board Structure & Evidence	20
B4.4 Decisions Log	20
B4.5 Wiki Documentation Index	22
Appendix A - Sticky-Note Colour Legend	23
Appendix B - Full Backlog Items from Event Storming	24
BACK-01 - Optimistic Locking on BookCopy	24
BACK-02 - CatalogueACL for Real-Time Availability Check	24
BACK-03 - OverdueDetectionService Scheduled Job	24
BACK-05 - CopyReleased Domain Event & Catalogue Subscriber	24
BACK-08 - UserBorrowingHistory Read Model	25
BACK-10 - Fine/Penalty System Research Spike	25
BACK-12 - WebSocket Real-Time Club Chat	Error! Bookmark not defined.

B1. Event Storming Outcomes

Using a shared virtual Miro board, three Event Storming sessions were conducted. The full business timeline was mapped (Session 1), and this was followed by Sessions 2 & 3, which were Process-level sessions (Sessions 2 & 3) focusing on Lending and Book-Club sub-domains. All artefacts are colour-coded using the standard Event Storming palette, which is shown below.

B1.1 Notation Key Used on the Board

Domain Event (orange)	Command (blue)	Actor / User (yellow)	Policy / Rule (purple/lilac)	Read Model / View (green)	External System (pink)	Aggregate (pale yellow)	Hotspot / Issue (red)
---------------------------------	--------------------------	---------------------------------	--	-------------------------------------	----------------------------------	-----------------------------------	---------------------------------

B1.2 Big-Picture Event Flow (Session 1 - Full Domain Timeline)

The board was filled left-to-right in a chronological order. Each E-library pipeline is represented by a row. Commands, actors, policies, and hotspots are shown and are cross referenced to their respective events.

#	Domain Event (orange)	Triggering Command (blue)	Actor (yellow)	Policy / Rule (purple)	Hotspot / Conflict (red)
1	UserRegistered	Register User	Student / Staff	Email must be unique	
2	UserAuthenticated	Login	Any Role	Invalid credentials -> retry with backoff	How many retry attempts before lockout?
3	PasswordChanged	Change Password	Authenticated User	Must confirm current password first	
4	BookAdded	Add Book	Admin	ISBN must be unique; Admin role required	Who validates ISBN format - domain or UI?
5	BookUpdated	Update Book Details	Admin	Cannot reduce totalCopies below active loans	Concurrent update conflict possible

6	BookSearched	Search Books	Student / Staff	Filter by title, author, category, availability	
---	--------------	--------------	-----------------	---	--

7	CopyAvailabilityChecked	Check Availability	Student	Real-time copy count must be shown	Cache staleness - how fresh is availability data?
8	LoanCreated	Borrow Book	Student / Staff	Max 5 active loans; copy must be AVAILABLE	Race condition if two users borrow last copy simultaneously
9	DueDateAssigned	(system - on Borrow)	System	Due date = issueDate + 14 days by default	Should Staff have a different loan period?
10	BorrowConfirmationSent	(triggered by policy)	Notification Svc	On LoanCreated -> send confirmation within 30 secs	
11	DueSoonReminderSent	(scheduled trigger)	Notification Svc	3 days before due date; send once per loan per window	What channel - email, in-app, or both?
12	LoanOverdue	(scheduled trigger)	System Scheduler	On due date if not returned -> flag OVERDUE	No fine system currently - is one needed?
13	OverdueAlertSent	(triggered by policy)	Notification Svc	On LoanOverdue -> notify user and staff	
14	BookReturned	Return Book	Student / Staff	returnDate >= issueDate; set LoanStatus = RETURNED	
15	CopyReleased	(triggered by policy)	System	On BookReturned -> restore copy	

				AvailabilityStatus	
16	ReturnConfirmationSent	(triggered by policy)	Notification Svc	On BookReturned - > confirm return to user	
17	BookClubCreated	Create Book Club	Any Authenticated User	Creator becomes OWNER automatically	
18	MemberJoined	Join Club	Student / Staff	Cannot join same club twice	Should clubs be moderated/inviteonly?
19	MessagePosted	Post Message	Club Member	Content <= 2000 chars; must be club member	Real-time delivery vs polling - which approach?
20	MemberRemoved	Remove Member	OWNER / MODERATOR	OWNER cannot remove themselves	
21	BookDeleted	Delete Book	Admin	Cannot delete if active loans exist for this book	Should deletion be soft-delete?

B1.3 Hotspots & Conflicts Register

Using red stickies, certain hotspots and conflicts were raised during the session. These sessions were marked for resolution. In the table below, each is tagged with its resolution status.

ID	Hotspot Description	Context	Resolution / Decision	Status
H-01	Race condition: two users borrow the last available copy at the same time	Lending	LoanIssuanceService uses optimistic locking on the BookCopy aggregate; The second request gets HTTP 409 Conflict response with user-friendly message	RESOLVED

H-02	Cache staleness: availability data shown to user may be out of date by seconds	Catalogue	Accept eventual consistency for search results; final availability check is done in real-time at time of borrowing (synchronous ACL call)	RESOLVED
H-03	Retry lockout: no policy defined for repeated failed login attempts	Identity & Access	Add domain policy: 15 minute account lock if 5 failed attempts within 10 minutes. Added to backlog as BACK-07	
H-04	ISBN validation: should the domain or UI layer validate ISBN format?	Catalogue	Domain validates via ISBN Value Object constructor. Prevalidation UI provides client-side for UX only. Domain is authoritative.	RESOLVED
H-05	Concurrent book update: admin updates totalCopies while borrow is in flight	Catalogue / Lending	Domain invariant: totalCopies >= activeLoans enforced in InventoryService. Surface written as optimistic by conflicting as optimistic locking exception.	
H-06	Staff loan period: should Staff get a longer default loan period than Students?	Lending	Product decision: Staff gets 28-day default; Student 14-day default. Role parameter accepted by DueDate VO factory method. Added to backlog BACK-08	RESOLVED

H-07	Notification channel: email only, in-app only, or both?	Notification	Initial release: notifications in-app only. Email integration deferred to v2. Backlog item BACK-09 created.	DEFERRED
H-08	Fine system: should overdue books incur a penalty / fine?	Lending	Out of scope of project. Recorded as future feature BACK-10.	DEFERRED
H-09	Book club moderation: should clubs be openjoin, or invite/approval required?	Book Club	Default open-join. OWNER can toggle club to invite-only mode. Added to backlog BACK-11.	DEFERRED
H-10	Real-time messaging: WebSocket vs polling for club chat	Book Club	WebSocket (STOMP over SockJS) chosen for real-time feel. Event driven style used by Chat service internally. Added to backlog BACK-12.	RESOLVED
H-11	Soft delete: should Book deletion be hard or soft (archive)?	Catalogue	Soft-delete chosen: deletedAt timestamp added to Book entity. Loan history references remain valid. Added to domain model.	RESOLVED

B1.4 Policies Discovered

Purple stickies, which represent policies, are automatic responses to events within the domain. For example: Do Y, when X happens. After discussions in the session, these policies are now written down as domain rules or application-level handlers.

Policy ID	Trigger Event	Policy Statement	Implemented In
POL-01	LoanCreated	Immediately send a BorrowConfirmationSent notification to the borrowing user when a loan is created.	NotificationDispatchService (async event handler)

POL-02	LoanCreated	When a Loan is created, mark the corresponding BookCopy as BORROWED (AvailabilityStatus)	LoanIssuanceService (synchronous, within Lending context)
POL-03	Scheduled (daily)	Each day, check all active Loans: if dueDate is exactly 3 days away and reminder not yet sent, send DueSoonReminderSent	OverdueDetectionService (scheduled job, @Scheduled annotation)
POL-04	Scheduled (daily)	Each day, check all active Loans: transition LoanStatus to OVERDUE if dueDate has passed, and fire LoanOverdue	OverdueDetectionService (scheduled job)
POL-05	LoanOverdue	When LoanOverdue fires, dispatch an OverdueAlertSent notification to the user, and to Staff, a summary alert.	NotificationDispatchService (async event handler)
POL-06	BookReturned	When a Book is returned, set LoanStatus = RETURNED, set returnDate, and fire CopyReleased	ReturnProcessingService (synchronous, within Lending)
POL-07	CopyReleased	When a CopyReleased event is received in Catalogue, update BookCopy.AvailabilityStatus = AVAILABLE	CatalogueEventHandler (async, Catalogue context)
POL-08	BookReturned	When a Book is returned, dispatch a ReturnConfirmationSent notification	NotificationDispatchService (async event handler)
POL-09	UserRegistered	When a new user registers, send a welcome notification	NotificationDispatchService (async event handler)
POL-10	MemberJoined	When a user joins a Book Club, send a welcome club notification	NotificationDispatchService (async event handler)
POL-11	BookDeleted (Admin)	When an Admin deletes a Book, check for active loans, and reject command and return domain error if any exist.	BookManagementService (precondition guard)

B2. Translation of Insights

The section below shows how (a) updated domain model elements, (b) policies, or (c) product backlog items are linked to the relevant stakeholder insight and hotspot resolution.

It shows how each significant stakeholder insight and hotspot resolution is connected to an updated domain model element, a policy, or a product backlog item. Each Delta is explained and commits reference the relevant GitHub issue or branch.

B2.1 Model Changes Introduced by Event Storming

The table below shows every change to the domain model that was made during Event Storming sessions, compared to Assignment 3 Part A.

Delta ID	Change Type	Element Changed	Before (Part A Baseline)	After (Post-Event Storming)	Traceable To
D-01	New Value Object	DueDate (Lending)	Modelled as plain Date field on Loan	Extracted into DueDate VO; factory method accepts Role to set 14day (Student) or 28-day (Staff) window; exposes isOverdue(): Boolean and daysUntilDue(): Int	H-06, POL-03, POL-04
D-02	New Domain Event	CopyReleased (Lending ->Catalogue)	No explicit release event; availability updated synchronously inside Lending	Explicit CopyReleased domain event published by ReturnProcessingService ; Catalogue subscribes asynchronously via ACL	POL-07, B1 flow step 15
D-03	Invariant Added	Book.deletedAt (Catalogue)	Hard-delete: row removed from DB	Soft-delete: deletedAt DateTime [0..1] added to Book entity; for search, deleted books should be hidden but preserved for loan history FK integrity	H-11, POL-11
D-04	New Policy Object	AccountLockPolicy (Identity & Access)	No lockout policy defined	New domain rule: LockoutPolicy VO encapsulates maxAttempts (5), windowMinutes (10), lockDurationMinutes (15); enforced in AuthenticationService	H-03, BACK-07

D-05	Aggregate Change	ClubMembership.inviteRequired (Book Club)	ClubMembers hip had no invite flag	BookClub gains inviteOnly: Boolean attribute; MemberJoined command path branches based on this flag; invite-only clubs, new MembershipRequestCreated event added for	H-09, BACK-11
D-06	New Domain Service	OverdueDetectionService (Lending)	No explicit scheduled service; described informally	Formalised as a Domain Service with clear interface: detectAndMarkOverdue(), sendRemindersBefore(days: Int); owns the LoanOverdue event publication	POL-03, POL-04, H-06
D-07	New Domain Event	MembershipRequestCreated (Book Club)	Did not exist	When user requests to join an invite-only club, a new event is published; OWNER/MODERATOR must approve via ApproveMembership command	D-05, H09
D-08	Multiplicity Update	Notification -> User association	0..* Notifications received by User (conceptual only)	Notification.userId is a reference key; NotificationRepository exposes findByUserId(paginated) for user inbox read model	B1 flow steps 10-13, 16
D-09	New Read Model	UserBorrowingHistory (Lending)	No explicit read model; raw Loan query only	Dedicated read-model projection: for efficient dashboard rendering, UserBorrowingHistory aggregates Loan records per userId; updated on LoanCreated/BookReturned events	B1 flow, Domain Storytelling DS-01

D-10	Invariant Tightened	Loan max active per user	Described as max 5 in SRS; not modelled as a VO	MaxActiveLoansPolicy VO (value: 5) added into LoanIssuanceService; configuration-driven so code change isn't needed for value change	H-08 (related), POL-02
------	---------------------	--------------------------	---	--	------------------------

B2.2 Backlog Items Generated

A traceable backlog item was produced for each unresolved hotspot and every model delta that was produced. The table below shows critical items introduced or updated during the Event Storming sessions. The full backlog is in Appendix B.

Backlog ID	Title	Type	Priority	Origin
BACK-01	Implement optimistic locking on BookCopy to prevent double-borrow race condition	Bug / Invariant	HIGH	H-01
BACK-02	Build LoanIssuanceService with real-time copy availability check via CatalogueACL	Feature	HIGH	H-01, H-02
BACK-03	Implement OverdueDetectionService as a @Scheduled job in the Lending microservice	Feature	HIGH	POL-03, POL-04, D-06
BACK-04	Implement soft-delete for Books (add deletedAt, filter in search queries)	Feature	HIGH	H-11, D-03
BACK-05	Implement CopyReleased async event and Catalogue subscriber	Feature	HIGH	D-02, POL-07
BACK-06	Extract DueDate Value Object with role-based factory (14-day Student / 28-day Staff)	Refactor / Feature	HIGH	H-06, D-01
BACK-07	Implement AccountLockPolicy (5 attempts / 10-min window / 15-min lock)	Security Feature	HIGH	H-03, D-04
BACK-08	UserBorrowingHistory read-model projection for the dashboard	Feature	MEDIUM	D-09
BACK-09	Email notification channel integration (v2 scope)	Feature	LOW	H-07

BACK-10	Investigate fine/penalty system for overdue loans (future feature)	Research	LOW	H-08
BACK-11	Implement invite-only Book Club mode with membership request/approval flow	Feature	MEDIUM	H-09, D-05, D-07
BACK-12	Implement WebSocket (STOMP/SockJS) for real-time club chat	Feature	MEDIUM	H-10

B3. Domain Storytelling

Using structured narratives, domain stories were used to capture how real actors collaborate to achieve goals in the domain. The format followed by each story: Actor -> [does something with] -> Work Object -> [to/from] -> Other Actor. These stories were created with all team members acting as domain expert proxies. This occurred during Session 3, and was validated against the event flow discovered in Session 1.

B3.1 DS-01: Student Borrows a Book

Actors: Student, E-Library System, Lending Service, Notification Service

Trigger: Student wants to borrow an available book

Outcome: Loan record created; student receives confirmation; copy marked unavailable

Step	Scene	Actor	Action	Work Object / System Response	Domain Term Reinforced
1	Authentication	Student	Navigates to the login page and university credentials submission	Login Form (email + password)	UserAccount, HashedPassword
2	Authentication	E-Library System	Validates credentials against the UserAccount aggregate; issues a JWT	JWT Token (contains userId, role=STUDENT)	AuthenticationService, JWT, Role
3	Authentication	E-Library System	Redirects student to their personalised User Dashboard	User Dashboard (shows active loans, search bar)	Read Model: UserBorrowingHistory
4	Book Search	Student	Types a book title into the search bar and submits	Search Query (title, category filter)	BookSearchService, Category

5	Book Search	Catalogue Service	Executes search; matching Book records with live availability counts returned	Search Results (title, author, availableCopies)	BookCopy, AvailabilityStatus
6	Book Selection	Student	Identifies the desired book and clicks 'Borrow'	Borrow Command (bookId, userId)	Command: BorrowBook
7	Availability	Lending Service	Calls CatalogueACL to verify that at least one BookCopy is AVAILABLE	CopyAvailabilityResponse (copyId, status=AVAILABLE)	CatalogueACL, AvailabilityStatus
8	Loan Creation	Lending Service	Creates Loan aggregate; assigns DueDate based on Role (Student = +14 days)	Loan (loanId, issueDate, dueDate, ACTIVE)	Loan, DueDate VO, LoanIssuanceService
9	Copy Update	Lending Service	Fires LoanCreated event; BookCopy updated to BORROWED by Catalogue ACL	CopyReleased / AvailabilityStatus = BORROWED	Domain Event: LoanCreated, Policy POL-02
10	Notification	Notification Service	Consumes LoanCreated event; dispatches BorrowConfirmation Sent to the student	In-app Notification (loan details, due date)	NotificationDispatcherService, Policy POL-01
11	Confirmation	Student	Confirmation notification received in their dashboard; sees book in Active Loans	Updated Dashboard (loan listed, due date visible)	LoanStatus = ACTIVE, Read Model

Ubiquitous Language Improvements from DS-01

- 'Borrow' replaces 'issue' in all user-facing language, now students 'borrow' books, the system 'issues' loans internally. Both terms now have distinct definitions in the glossary.
- 'Due Date' is now always expressed as a Date (not DateTime) in the UI, the time component is hidden from users to avoid confusion.
- 'Available Copies' is the user-facing term; internally the domain uses AvailabilityStatus on individual BookCopy instances.

B3.2 DS-02: Admin Manages Book Inventory

Actors: Admin, E-Library System, Catalogue Service

Trigger: New books have arrived; Admin must add them to the system

Outcome: Books available for search and borrowing; notifications sent if staff are watching

Step	Scene	Actor	Action	Work Object / System Response	Domain Term Reinforced
1	Authentication	Admin	Logs in; JWT contains role=ADMIN	JWT (role=ADMIN)	Role, RBAC enforcement
2	Navigation	Admin	Opens the Admin Book Management panel	Admin Dashboard (restricted route)	RBAC protected route
3	Book Entry	Admin	Fills in book form: ISBN, title, description, authors, categories, copy count	AddBook Command (all fields)	Command: AddBook
4	ISBN Validation	Catalogue Service	ISBN VO constructor validates format (10 or 13 digits); domain rejects invalid	Domain Validation Result	ISBN Value Object, invariant
5	Persistence	Catalogue Service	Creates Book entity and n BookCopy instances; sets all copies of books to AVAILABLE	BookAdded Event (bookId, isbn, title, copies)	Domain Event: BookAdded, BookCopy
6	Inventory	Catalogue Service	InventoryService sets availableCopies = N; updates totalCopies	Book record (totalCopies=N, availableCopies=N)	InventoryService, invariant

7	Confirmation	E-Library System	Returns success response; admin book list adds book immediately	Updated Admin Book List	Read Model
8	Update flow	Admin	Later updates book description with Edit Book form	UpdateBook Command (bookId, new description)	Command: UpdateBook
9	Delete guard	Admin	Attempts to delete a book that has 2 active loans	DeleteBook Command (bookId)	Command: DeleteBook, POL-11
10	Reject	Catalogue Service	Domain check fails: active loans exist ->	Domain Error: 'Cannot delete	Invariant enforcement, soft-delete
			descriptive error accompanies rejected command	book with active loans'	
11	Soft delete	Admin	Chooses to archive the book instead; sets deletedAt	BookDeleted Event (soft) - book hidden from search	Domain Event: BookDeleted, D03

Ubiquitous Language Improvements from DS-02

- 'Archive' vs 'Delete': there is now a clear distinction between archiving (soft-delete, history preserved) and deletion. The term 'Archive Book' is added to the glossary.
- 'Book Copies' and 'Books' are now explicitly separate in all documentation, now users search for Books, but borrow Copies of said Book.

B3.3 DS-03: Overdue Book - Reminder and Return Flow

Actors: System Scheduler, Student, Staff, Notification Service

Trigger: 3 days before due date, then due date passes without return

Outcome: Reminders sent; loan flagged overdue; staff alerted; book eventually returned

Step	Scene	Actor	Action	Work Object / System Response	Domain Term Reinforced
1	Scheduled scan	System Scheduler	OverdueDetectionService runs daily; queries Loans where dueDate = today + 3 days and reminder not yet sent	Loan query result (active loans approaching due date)	OverdueDetectionService, DueDate VO
2	Reminder trigger	System Scheduler	Qualifying Loan, fires DueSoonReminderSent event for each	DueSoonReminderSent Event (loanId, userId, dueDate)	Policy POL-03, idempotency check
3	Notification	Notification Service	Consumes event; dispatches reminder to the student in-app	In-app Notification: 'Your book is due in 3 days'	NotificationDispatchService, NotificationStatus
4	Student response	Student	Ignores the reminder (book not returned by due date)	(No action taken)	Actor inaction modelled as a domain trigger
5	Overdue scan	System Scheduler	OverdueDetectionService runs on due date; identifies the Loan as overdue	LoanOverdue Event (loanId, userId, dueDate)	Policy POL-04, LoanStatus = OVERDUE
6	Status update	Lending Service	Sets LoanStatus = OVERDUE on the Loan aggregate	Updated Loan (status=OVERDUE)	LoanStatus VO, Loan invariant
7	Alert	Notification Service	Consumes LoanOverdue event; sends OVERDUE_ALERT to student and Staff receive a summary	Two notifications: student alert + staff summary	Policy POL-05, NotificationType
8	Staff view	Staff	Logs in; views overdue loan report in Staff dashboard	Staff Dashboard - overdue loans list	Role = STAFF, readmodel
9	Return	Student	Later, app Return action returns the book	ReturnBook Command (loanId)	Command: ReturnBook

10	Processing	Lending Service	ReturnProcessingService: sets returnDate, LoanStatus = RETURNED, fires CopyReleased	BookReturned Event, CopyReleased Event	ReturnProcessingService, Policy POL-06
11	Availability	Catalogue Service	CatalogueEventHandler receives CopyReleased; updates BookCopy to AVAILABLE	BookCopy (status=AVAILABLE)	Policy POL-07, CatalogueACL
12	Confirmation	Notification Service	Dispatches ReturnConfirmationSent notification to student	In-app Notification: 'Return confirmed'	Policy POL-08

Ubiquitous Language Improvements from DS-03

- 'Overdue' is now a formal LoanStatus value. It was used informally in descriptions but not modelled previously. The single point of truth for this transition is OverdueDetectionService.
- 'Reminder' vs 'Alert': Both terms are now distinct in the ubiquitous language. A Reminder (DUE_SOON_REMINDER) is proactive and friendly in tone; an Alert (OVERDUE_ALERT) is reactive and urgent in tone.
- The term 'return' is now consistently used for the actor action; the system internally says 'CopyReleased' - both terms coexist with clear ownership.

B3.4 DS-04: Book Club - Create, Join, and Discuss

Actors: Authenticated User, Club Owner, Moderator, Notification Service

Trigger: A student wants to discuss a book with peers

Outcome: Club created; members join; real-time messages exchanged

Step	Scene	Actor	Action	Work Object / System Response	Domain Term Reinforced
1	Create club	Authenticated User	Navigates to Book Clubs section; clicks 'Create Club'; fills in name and description	CreateClub Command (name, description, inviteOnly flag)	Command: CreateClub

2	Club creation	Book Club Service	Creates BookClub entity; creates ClubMembership for creator with ClubRole = OWNER	BookClubCreated Event (clubId, name, ownerId)	Domain Event: BookClubCreated, ClubRole = OWNER
3	Discovery	Another Student	Searches for clubs; finds the new club; clicks 'Join'	JoinClub Command (clubId, userId)	Command: JoinClub
4	Open join	Book Club Service	Club is open-join (inviteOnly=false); creates ClubMembership with ClubRole = MEMBER	MemberJoined Event (clubId, userId)	ClubMembership, Policy: cannot join twice
5	Welcome notification	Notification Service	Consumes MemberJoined event; new member gets sent welcome club notification	In-app Notification: 'Welcome to [Club Name]'	Policy POL-10
6	Discussion	Club Member	Opens club discussion page; types a message; submits	PostMessage Command (clubId, content)	Command: PostMessage, MessageContent VO
7	Message persist	Book Club Service	MessageService validates content (<= 2000 chars, non-empty); persists Message	MessagePosted Event (messageId, clubId, authorId)	Domain Event: MessagePosted, MessageService
8	Real-time	E-Library System	WebSocket (STOMP) pushes message to all connected club members in real-time	WebSocket push to club subscribers	Chat Service, eventdriven style

9	Moderation	Club Owner	Spots inappropriate message; uses Remove Message action	DeleteMessage Command (messageld)	ClubModerationService, soft-delete
10	Role change	Club Owner	Promotes an active member to MODERATOR role	ChangeRole Command (membershipId, newRole=MODERATOR)	ClubRole, ClubModerationService

Ubiquitous Language Improvements from DS-04

- 'Book Club' is the clear primary term not 'Reading Group' or 'Discussion Forum'. Book Club is described as a user-created community centred on a shared reading interest. The glossary is updated to reflect this.
- 'Owner' and 'Moderator' are now formal ClubRole values, and are used consistently in UI labels, API responses, and domain documentation.
- 'Discussion' refers to all Messages within a club. Internally represented as Message entities, which the UI shows on the 'Discussion' tab

B4. Facilitation & Evidence

In this section is documented the facilitation approach, session records, participant lists, decision log. As well as inks to the boards providing the evidence for the Event Storming and Domain Storytelling outputs above.

B4.1 Session Summary

Session	Date	Duration	Format	Participants	Facilitator	Focus
Session 1 - Big Picture Event Storming	Week 8, Tuesday	2.5 hours	Virtual Miro Board	All 6 team members	Oscar Zhou	Full domain timeline; event identification; hotspot tagging
Session 2 - ProcessLevel: Lending	Week 8, Thursday	1.5 hours	Virtual Miro Board	Caylum, Vivian, Daragh D., Darragh Q.	Caylum Hurley	Borrow/return/overdue flow; LoanIssuanceService design; H-01, H-02, H-06 resolution
Session 3 - Domain Storytelling	Week 9, Tuesday	2 hours	Discord + Miro	All 6 team members	Darragh Quinn	Four domain stories; ubiquitous language review; backlog refinement

B4.2 Participant Roles in Sessions

Team Member	Role in Sessions	Domain Expert Area
Caylum Hurley	Session 1 Facilitator; Context Map owner; notation enforcement	Book Club context; architecture style
Joe Considine	Domain expert proxy for Identity & Access; ACL contracts; policy writing	Authentication flows; role-based access
Oscar Zhou	Domain expert proxy for Catalogue; Hotspot H-04, H-05, H-11 owner	Book inventory; ISBN; soft-delete design
Daragh Downes	Session 2 Facilitator; Lending domain expert; H-01, H-02, H-06 owner	Loan lifecycle; overdue detection; DueDate VO
Vivian Jently	Session 3 Facilitator; Notification domain expert; backlog refinement	Event-driven notification policies; domain storytelling DS-03
Darragh Quinn	Book Club domain expert; WebSocket design (H-10); BACK-12 owner	Book Club context; real-time messaging; moderation

B4.3 Miro Board Structure & Evidence

There features five sections on the Miro board, with a horizontal timeline at the top. Each section represents a single domain context. The boards includes the following items:

- Domain Events timeline (orange stickies) - complete sequential flow of events, Sessions 1 and 2
- Commands (blue stickies) - one per event, aligned vertically with actor that triggers it
- Actors (yellow stickies) - Student, Staff, Admin, System, Notification Service
- Policy stickies (purple) - all 11 policies from Section B1.4 are posted above the event they apply to
- Hotspot stickies (red) - 11 hotspots marked, with resolution notes added in green during Sessions 2 and 3
- Aggregate boundaries (large pale-yellow frames) - UserAccount, Loan, Book, BookClub, Message
- Domain story diagrams (Session 3 panel) - pictogram-style DS-01 through DS-04

B4.4 Decisions Log

Where all notable decisions made during sessions are recorded for traceability. Delta IDs and Backlog IDs are used to refer to decisions that changed the domain model.

Decision ID	Decision Made	Session	Rationale	Model / Backlog Impact
DEC-01	Use optimistic locking, not pessimistic locking, to resolve the last-copy race condition	Session 2	Pessimistic locking blocks concurrent read, slowing search performance. Optimistic locking with retry is good for borrowing patterns with few conflicts	BACK-01
DEC-02	Accept eventual consistency for catalogue search results; only check in real-time at borrow time	Session 2	Synchronous Catalogue calls on every query needed for real-time availability on every search result page. Slight UX staleness is fine; correctness is enforced at borrow.	H-02, BACK-02
DEC-03	Differentiate loan periods by Role: 14 days for Student, 28 days for staff	Session 2	Request from staff raised during session. Maps to real library practice. To keep the change local, made with DueDate VO factory method.	H-06, D-01, BACK-06
DEC-04	Use soft-delete for Books to keep integrity of loan history	Session 1	When hard deleting a book, loan records would be left with dangling foreign keys. Softdelete (deletedAt) removes book from active search but keeps historical records valid.	H-11, D-03, BACK-04
DEC-05	Notification v1 scope: in-app only; email integration to come in v2	Session 1	Email integration requires more SMTP setup. and more required for email integration. For an MVP, in-app notifications deliver the core requirement. Email added to backlog as low priority.	H-07, BACK-09
DEC-06	WebSocket (STOMP over SockJS) allows to chat with others in club in real time	Session 3	Latency would be introduced by polling in club discussions. Spring supports STOMP natively, which works with the Spring Boot backend. SockJS gives backup option	H-10, D-05, BACK-12

			for environments without WebSocket support.	
DEC-07	Book Club invite-only mode: flag on BookClub entity; default is open-join	Session 3	The open-join default reduces friction for the MVP. Need for invite-only mode is a clear for users (discussed in DS-04 session). Set up as a boolean with new event path: MembershipRequestCreated.	H-09, D-05, D-07, BACK-11
DEC-08	AccountLockPolicy modelled as a Value Object, not a hardcoded constant	Session 2	Policies based on configuration are easier to test and maintain. The VO can change through configuration without changing the domain logic and is injected into AuthenticationService.	H-03, D-04, BACK-07

B4.5 Wiki Documentation Index

Event Storming sessions led to the creation or updating of the following Wiki pages. The “Domain Discovery” section of the project Wiki home page features links to all pages.

Wiki Page	Content	Last Updated	Owner
Event Storming Board	Embedded Miro board + PNG files exported from each session; session dates and attendance	Week 9	Oscar Zhou
Domain Event Catalogue	Complete list of 21 domain events along with payload schemas, publisher, and subscriber context	Week 9	Darragh Quinn
Policy Register	All 11 policies with trigger event, effect, and implementation location	Week 9	Vivian Jently

Hotspot Resolution Log	Log all 11 hotspots with status, resolution decision, and links to backlog	Week 9	Caylum Hurley
Ubiquitous Language Glossary	Updated glossary including all terms that came up in DS-01 through DS-04	Week 9	All members
Domain Stories	Four domain stories in pictogram and narrative format; linked to model deltas	Week 9	Darragh Quinn
Backlog - Event Storming Items	GitHub Project board milestone: 'Event Storming Discoveries' (BACK01 to BACK-12)	Week 9	Joe Considine
Model Delta Log	Tracked changes D-01 to D-10 with before/after descriptions and delta rationale	Week 9	Daragh Downes

Appendix A - Sticky-Note Colour Legend

The colour coding below was used across all Event Storming sessions and is used throughout this document.

Colour	Sticky Type	Description	Example
Orange	Domain Event	Something that happened in the domain; named in past tense	LoanCreated, UserRegistered, BookReturned
Blue	Command	An instruction to make something happen	BorrowBook, AddBook, PostMessage
Yellow	Actor	A person or system that starts a Command or responds to an Event	Student, Admin, System Scheduler
Lilac/Purple	Policy / Rule	An business rule that happens automatically; “when Event X happens, do Command Y”	POL-01: On LoanCreated -> send confirmation
Green	Read Model / View	A view or query result that is shown to a user; supports decision-making	User Dashboard, Search Results, Admin Book List
Pink	External System	A system outside our domain that interacts with it	Email SMTP (v2), University Auth SSO (future)
Pale Yellow	Aggregate	A dashed frame grouping an Aggregate Root and its children	Loan, BookClub, UserAccount

Red	Hotspot / Issue	A question, conflict, or uncertainty requiring resolution; marked H-xx	H-01: Race condition on last copy
-----	-----------------	--	-----------------------------------

Appendix B - Full Backlog Items from Event Storming

Below are twelve backlog items along with all creation information for GitHub issues. During Event Storming sessions, these items were added to, or changed in the backlog.

BACK-01 - Optimistic Locking on BookCopy

Priority	Affected Contexts	Origin
HIGH	Lending, Catalogue	H-01, D-02

Add @Version field to BookCopy JPA entity. In LoanIssuanceService, catch OptimisticLockingFailureException and return HTTP 409 response with message “This Copy was already borrowed. Please try again later.”

BACK-02 - CatalogueACL for Real-Time Availability Check

Priority	Affected Contexts	Origin
HIGH	Lending	H-01, H-02

Implement CatalogueACL class in Lending context. On BorrowBook command, call GET /v1/catalogue/copies/{copyId}/availability synchronously before creating Loan. Translate response to internal CopyReference VO.

BACK-03 - OverdueDetectionService Scheduled Job

Priority	Affected Contexts	Origin
HIGH	Lending	POL-03, POL-04, D-06

Implement two @Scheduled methods to OverdueDetectionService: (1) detectReminders() runs each day at 08:00, finds loans that are due in 3 days, and if DueSoonReminderSent is not already sent, it sends one; (2) detectOverdue() runs daily at 00:01, finds loans that past due date, changes their status to OVERDUE, and fires LoanOverdue.

BACK-04 - Soft-Delete for Books

Priority	Affected Contexts	Origin
HIGH	Catalogue	H-11, D-03

Add deletedAt: DateTime? to Book entity. Override BookRepository search queries to leave out records where deletedAt IS NOT NULL. When Books has active loads, Admin UI shows “Archive” button instead of “Delete” button.

BACK-05 - CopyReleased Domain Event & Catalogue Subscriber

Priority	Affected Contexts	Origin
----------	-------------------	--------

HIGH	Lending, Catalogue	D-02, POL-07
-------------	--------------------	--------------

After successful return, ReturnProcessingService should publish CopyReleased event. To receive this event (through Spring ApplicationEvent or message bus) and update BookCopy, implement CatalogueEventHandler in the Catalogue context and update BookCopy.AvailabilityStatus to AVAILABLE.

BACK-06 - DueDate Value Object with Role-Based Factory

Priority	Affected Contexts	Origin
HIGH	Lending	H-06, D-01

Extract DueDate VO in Lending context. Add factory method: DueDate.forRole(issueDate, role) which returns issueDate + 14 days for STUDENT and issueDate + 28 days for STAFF. Update LoanIssuanceService to call this factory. Update Loan entity.

BACK-07 - Account Lockout Policy

Priority	Affected Contexts	Origin
HIGH	Identity & Access	H-03, D-04

Implement AccountLockPolicy VO: maxAttempts=5, windowMinutes=10, lockDurationMinutes=15. Inject into AuthenticationService. Keep track of failed attempts in a FailedLoginAttempt entity (userId, timestamp). After maxAttempts in window, set UserAccount.lockedUntil timestamp. Return HTTP 423 Locked with retry-after header.

BACK-08 - UserBorrowingHistory Read Model

Priority	Affected Contexts	Origin
MEDIUM	Lending	D-09

Create UserBorrowingHistory read-model projection. Subscribe to LoanCreated and BookReturned events; keep a denormalised table (userId, loanId, bookTitle, issueDate, dueDate, returnDate, status) that optimised for dashboard queries. Expose via GET /lending/users/{userId}/history (with pagination).

BACK-09 - Email Notification Channel (v2)

Priority	Affected Contexts	Origin
LOW	Notification	H-07

Implement JavaMailSender with Spring Boot to send emails. Use the NotificationChannel interface to create EmailNotificationAdapter. Configuration-driven: enable via application.properties. Deferred to v2 - do not block v1 release.

BACK-10 - Fine/Penalty System Research Spike

Priority	Affected Contexts	Origin
LOW	Lending	H-08

Research: would a fine system add improve the academic E-Library MVP? Look into how feasible it is, user acceptance, and how difficult the implementation will be. Output: one-page recommendation document in Wiki.